

Part 1: <https://studio.edgeimpulse.com/public/989450/live>

1. The raw inputs in this Edge Impulse project are accelerometer data in the X, Y, and Z directions.
2. Some feature types passed into the NN classifier are accelerometer RMS data, height of spectral peaks for an accelerometer axis, frequency of a peak for an accelerometer axis, and spectral power at different frequencies. There are 33 input features.. Five examples of feature names used to train the NN classifier are accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, and accY RMS.
3. The NN classifier outputs two classes: nominal and off.
4. Feature types passed to the anomaly detector were RMS data, peak heights, and spectral power. Four features were passed to the detector. The feature names were accX RMS, accX Peak 1 Height, accY Spectral Power 2.0 – 5.0 Hz, and accZ Spectral Power 2.0 – 5.0 Hz.

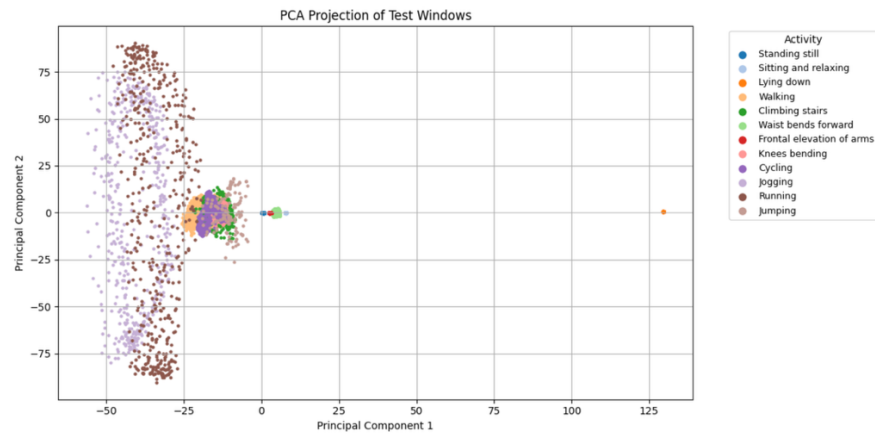
```
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 15.975330
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 527 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
5. Anomaly prediction: 16.022768
```

6. The deployment results show the confidence score for accelerometer data being a nominal reading or an off reading. There's also an anomaly prediction score showing how likely the reading is anomalous. The classifier output shouldn't always be trusted if the anomaly prediction score is high, or if the class predictions are similar to each other (it's uncertain if the fan is nominal or off). Some conditions that can make the classifier reliable is if the fan is running at a constant speed and the fan blade is balanced, meaning there's no extra vibrations. Some conditions that make the classifier unreliable could be if there's variable speeds or the fan is speeding up or slowing down. Sensor drift could also give unreliable data, which makes the models unreliable.

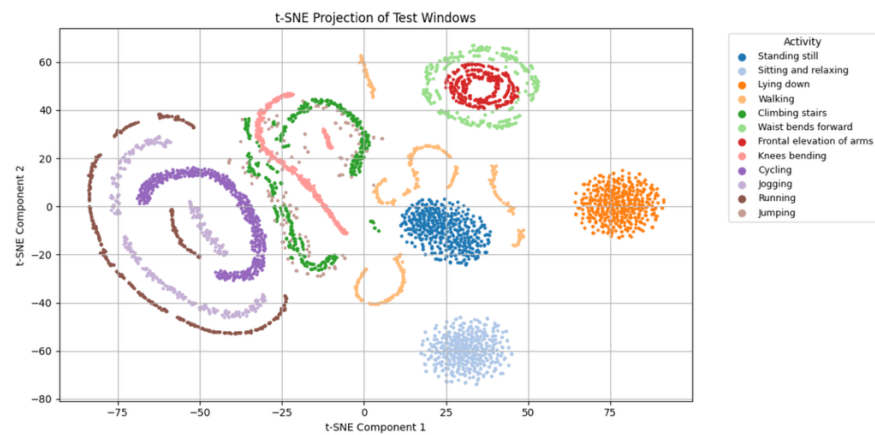
Part 2:

https://github.com/nhl01/EE446Labs/tree/main/Lab6_TinyML_Anomaly_Detection

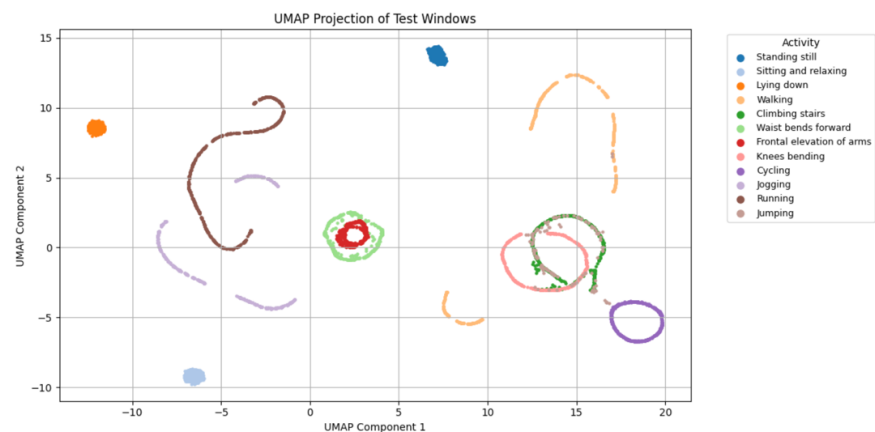
Autoencoder Model Results:



PCA Visualization

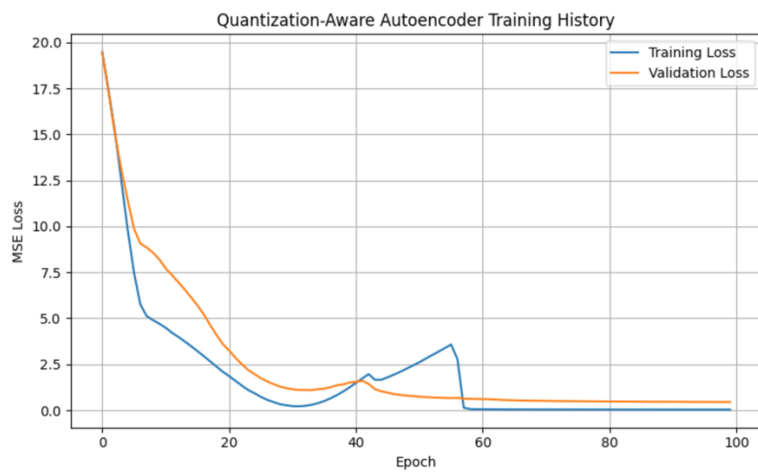
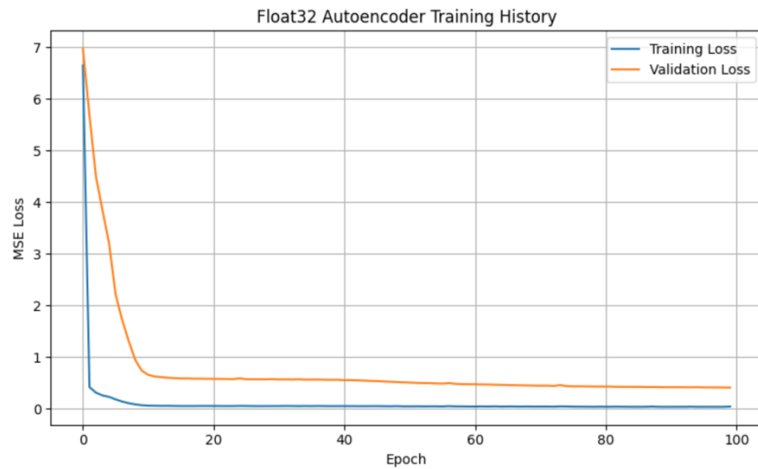


t-SNE Visualization

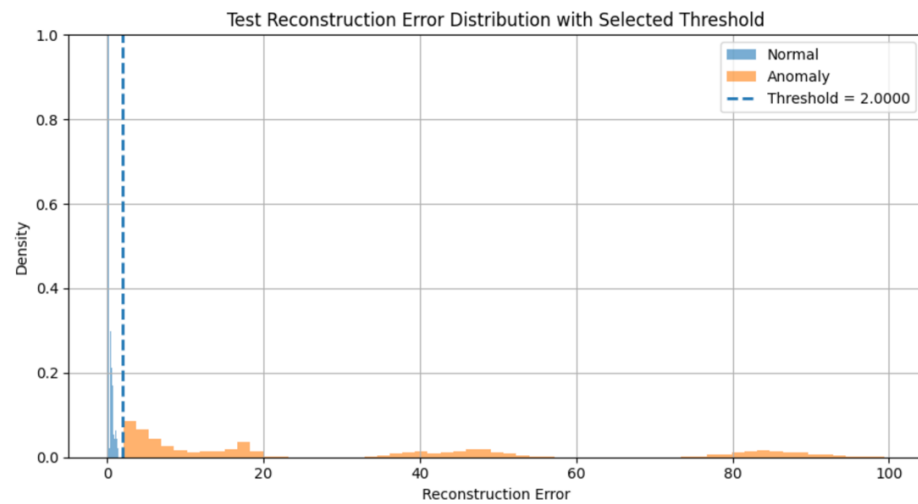


UMAP Visualization

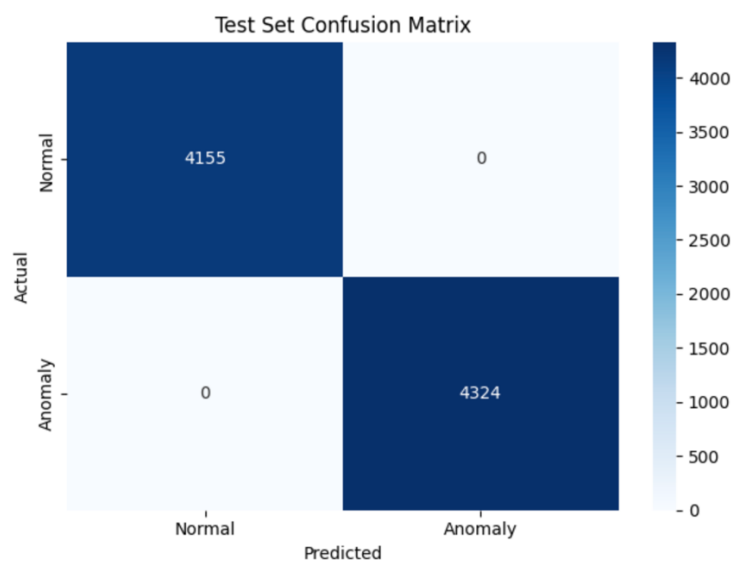
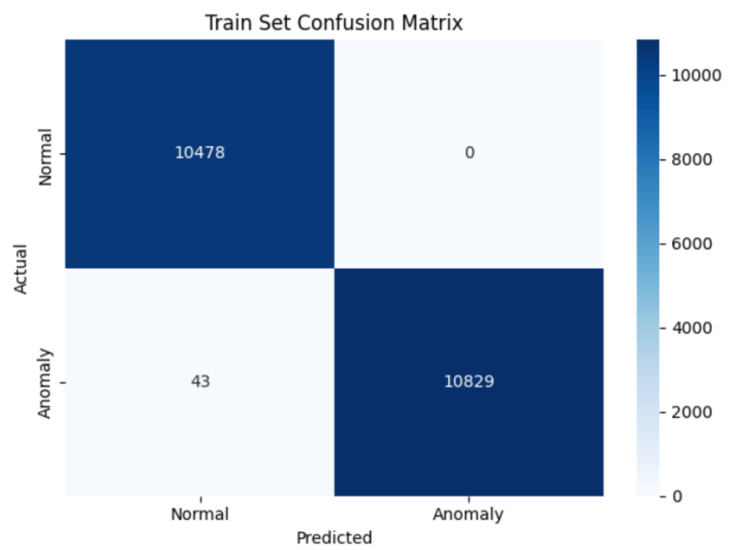
Nick Leung EE446 SP26 Lab 6 Report



Training and Validation Loss Curves



Reconstruction Error Distribution and Selected Anomaly Threshold



Confusion Matrices

Train set evaluation: int8 model				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	10478
Anomaly (1)	1.00	1.00	1.00	10872
accuracy			1.00	21350
macro avg	1.00	1.00	1.00	21350
weighted avg	1.00	1.00	1.00	21350
Test set evaluation: int8 model				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324
accuracy			1.00	8479
macro avg	1.00	1.00	1.00	8479
weighted avg	1.00	1.00	1.00	8479

Evaluation Metrics

```
Reconstruction error: 2.208680  
Result: Anomaly detected  
Reconstruction error: 2.203787  
Result: Anomaly detected  
Reconstruction error: 1.928631  
Result: Normal activity  
Reconstruction error: 1.789426  
Result: Normal activity  
Reconstruction error: 1.593445  
Result: Normal activity
```

Arduino outputs

Discussion Questions:

I picked a threshold of 2 for anomaly detection. When visually inspecting my error distribution, 2 is between the high boundary of the normal class's distribution and the low boundary of the anomaly class's distribution.

For an autoencoder, the reconstructed data should usually match the input data closely. The autoencoder is trained only on normal data, so it can reconstruct those inputs well. If the reconstructed data is too different from the input data (the error is too high), then the input was anomalous. Picking a threshold for the reconstruction error helps determine if a certain error value corresponds to an anomaly.

The Python notebook needs to preserve the `autoencoder_model.cc` file, the reconstruction error threshold, window size, stride, and quantization parameters (scale and zero point) when deploying to the Arduino. The file stores the model's weights as bytes and the threshold determines what is normal vs. anomalous. Consistent window size and stride makes sure the input shape to the model is correct. Storing the correct quantization parameters helps the model perform inference on proper inputs.

It's important for the Arduino sketch to use the same preprocessing assumptions as the notebook because any discrepancies would mean the Arduino will send out-of-range data to the model compared to the notebook, resulting in unreliable outputs. The Arduino needs to pre-process the raw sensor data the same way the notebook pre-processes the data so the ranges and distributions match. One example is converting from g-forces (sensor data units) to meters per second squared (dataset units).

The main limitations of this anomaly detection method when deployed on a microcontroller could be a small window size not correctly capturing very quick anomalous events, failure to account for sensor drift, and not generalizing the data to other people. We were only given data for one subject out of ten test subjects.